



Libasm

Assembly yourself!

Summary: The aim of this project is to become familiar with assembly language.

Version: 6.0

Contents

I	Foreword	2
II	Introduction	3
III	Common Instructions	4
IV	AI Instructions	5
V	Mandatory part	7
VI	Bonus part	8
VII	Submission and peer-evaluation	9
Annexes		10
VII.1	ft_atoi_base	10
VII.2	ft_list_push_front	11
VII.3	ft_list_size	12
VII.4	ft_list_sort	13
VII.5	ft_list_remove_if	14
VII.6	ft_simd_memchr	15

Chapter I

Foreword

In 1947, Kathleen Booth was writing out instructions for the ARC, one of the first computers built at Birkbeck College, London, by hand, as strings of binary code. To make that work bearable, she designed a notation of short mnemonics standing for each operation, along with an assembler to translate that notation back into the machine's raw instructions. That notation became one of the very first assembly languages, and the program that read it, one of the first assemblers. She went on to write the assemblers for the ARC2 and the SEC, and to co-author one of the earliest textbooks on programming these machines.

Every time you write a mnemonic instead of an opcode and let a tool turn it into bytes, you are using the idea she built. This project asks you to work at the level she worked at: close enough to the machine to see exactly what it does, with a notation that makes that work possible.

Chapter II

Introduction

An assembly (or assembler) language, often abbreviated asm, is a low-level programming language for a computer, or other programmable device, in which there is a very strong (but often not one-to-one) correspondence between the language and the architecture's machine code instructions. Each assembly language is specific to a particular computer architecture. In contrast, most high-level programming languages are generally portable across multiple architectures but require interpreting or compiling. Assembly language may also be called symbolic machine code.

Chapter III

Common Instructions

- Your functions should not quit unexpectedly (segmentation fault, bus error, double free, etc.) except in case of undefined behavior. If this happens, your project will be considered non-functional, and you will receive a 0 during the evaluation.
- Your Makefile must at least contain the rules `$(NAME)`, `all`, `clean`, `fclean` and `re`. It must recompile/relink only the necessary files.
- To include bonuses in your project, you must add a `bonus` rule to your Makefile, which will incorporate all the various headers, libraries, or functions that are forbidden in the main part of the project. Bonuses must be in different files `_bonus.s`. The evaluation of the mandatory and bonus parts is done separately.
- We encourage you to create test programs for your project, even though this work **won't have to be submitted and won't be graded**. It will give you a chance to easily test your work and your peers' work. You will find those tests especially useful during your defence. Indeed, during defence, you are free to use your tests and/or the tests of the peer you are evaluating.
- Submit your work to your assigned Git repository. Only the work in the Git repository will be graded. If Deepthought is assigned to grade your work, it will be done after your peer-evaluations. If an error occurs in any section of your work during Deepthought's grading, the evaluation will stop.
- You must write 64-bit assembly. Beware of the "calling convention".
- You can't do inline ASM, you must do '.s' files.
- You must compile your assembly code with `nasm`.
- You must use the Intel syntax, not the AT&T syntax.



It is forbidden to use the compilation flag: `-no-pie`.

Chapter IV

AI Instructions

● Context

AI is now a powerful coding partner — alongside your peers — for tackling large and demanding projects. You will guide it through both technical and non-technical aspects of your work.

AI tools can boost your efficiency and improve the quality of your output, but you should be able to dive deep into any part of the project without relying on them.

Your AI partner supports you, but you remain fully responsible for making informed technical decisions and to clearly explain and defend them.

● Main message

- 👉 Strive for a mature and responsible use of AI.
- 👉 Never let AI take responsibility for decisions — especially when it lacks awareness of your goals, constraints, or team dynamics.
- 👉 Maintain creativity, innovation, and human oversight through active collaboration with your peers. AI is trained on existing data and rarely generates truly new ideas.
- 👉 Stay informed about emerging trends and be ready to adapt to new concepts and technologies.

● Learner rules:

- Maintain intellectual leadership over your projects and make your own informed decisions.
- Prioritise the collective intelligence of your team and peers.
- Actively stay informed about the ongoing evolution of AI technologies.

● Phase outcomes:

- AI engineering skills.
- Increased efficiency.
- Greater reliability and quality.
- A pioneering mindset.

● Comments and examples:

- Your peers can identify trade-offs, question assumptions, and help you improve. The first answer from an AI might not be the best — it may lack efficiency, security, or real added value. Now more than ever, you should rely on your peers.
- AI can make you faster, but your peers make you better. Collaboration, discussion, and mutual challenge are key to success.
- Be transparent about how AI was used in your projects, and clearly identify what was generated by AI tools.

✓ Good practice:

I asked AI to help generate unit tests for my API. I reviewed them with my teammate, and we adjusted them for edge cases. It saved time, and we both learned something new.

✗ Bad practice:

I had AI generate the entire architecture of my project. It “works,” but when I’m asked to explain design decisions during the peer review or in front from of a customer, I cannot. I lose credibility and I fail.

Chapter V

Mandatory part

- The library must be called libasm.a.
- You must submit a main function that will test your functions and compile with your library to demonstrate that it is functional.
- You must rewrite the following functions in assembly:
 - ft_strlen (man 3 strlen)
 - ft_strcpy (man 3 strcpy)
 - ft_strcmp (man 3 strcmp)
 - ft_write (man 2 write)
 - ft_read (man 2 read)
 - ft_strdup (man 3 strdup, you can call to malloc)
 - ft_calloc (man 3 calloc, you can call to malloc; you must detect and handle a `size_t` overflow in `nmemb * size` yourself, instead of relying on malloc to catch it)
- You must check for errors during syscalls and handle them properly when needed.
- Your code must set the variable `errno` properly.
- For that, you are allowed to call the `extern` `___error` or `errno_location`.

Chapter VI

Bonus part

You can rewrite these functions in assembly. The linked list functions will use the following structure:

```
typedef struct    s_list
{
    void          *data;
    struct s_list *next;
}                t_list;
```

- ft_atoi_base (see Annex VII.1)
- ft_list_push_front (see Annex VII.2)
- ft_list_size (see Annex VII.3)
- ft_list_sort (see Annex VII.4)
- ft_list_remove_if (see Annex VII.5)
- ft_simd_memchr (see Annex VII.6)



The bonus part will only be assessed if the mandatory part is PERFECT. Perfect means the mandatory part has been integrally done and works without malfunctioning. If you have not passed ALL the mandatory requirements, your bonus part will not be evaluated at all.

Chapter VII

Submission and peer-evaluation

Submit your assignment to your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Do not hesitate to double-check the names of your folders and files to ensure they are correct.

Annexes

VII.1 ft_atoi_base

- Write a function that converts the initial portion of the string pointed to by `str` into an integer representation.
- `str` is in a specific base, given as a second parameter.
- Except for the base rule, the function should behave exactly like `ft_atoi`.
- If an invalid argument is provided, the function should return 0.
Examples of invalid arguments:

- The base is empty or has only one character.
- The base contains duplicate characters.
- The base contains `+`, `-`, or whitespace characters.

- The function should be prototyped as follows:

```
int ft_atoi_base(char *str, char *base);
```

VII.2 ft_list_push_front

- Create the function `ft_list_push_front`, which adds a new element of type `t_list` to the beginning of the list.
- It should assign `data` to the given argument.
- If necessary, it will update the pointer at the beginning of the list.
- You are allowed to call `malloc` to allocate the new element.
- Prototype:

```
void ft_list_push_front(t_list **begin_list, void *data);
```

VII.3 ft_list_size

- Create the function `ft_list_size`, which returns the number of elements in the list.
- Prototype:

```
unsigned int ft_list_size(t_list *begin_list);
```

VII.4 ft_list_sort

- Create the function `ft_list_sort` which sorts the list's elements in ascending order by comparing two elements and their data using a comparison function.
- Prototype:

```
void ft_list_sort(t_list **begin_list, int (*cmp)());
```

- The function pointed to by `cmp` will be used as:

```
(*cmp)(list_ptr->data, list_other_ptr->data);
```



`cmp` could be for instance `ft_strcmp`.

VII.5 ft_list_remove_if

- Create the function `ft_list_remove_if` which removes from the list all elements whose data, when compared to `data_ref` using `cmp`, causes `cmp` to return 0.
- The data from an element to be erased should be freed using `free_fct`.
- Once its data has been freed, the element itself must be freed using `free`.
- Prototype:

```
void ft_list_remove_if(t_list **begin_list, void *data_ref, int (*cmp)(), void (*free_fct)(void *));
```

- The functions pointed to by `cmp` and `free_fct` will be used as:

```
(*cmp)(list_ptr->data, data_ref);  
(*free_fct)(list_ptr->data);
```

VII.6 ft_simd_memchr

- Create the function `ft_simd_memchr`, which behaves like `memchr` (man 3 `memchr`): it locates the first occurrence of the byte `c` in the first `n` bytes of the memory area `s`, and returns a pointer to it, or `NULL` if `c` does not occur in those `n` bytes.
- This function must exploit data-level parallelism: each comparison instruction must test several bytes of `s` against `c` at once, instead of looping over the buffer one byte at a time. You can use either the SSE (or later) vector registers (`xmm0-xmm15`), or a SWAR (SIMD Within A Register) technique packing several bytes into a single general-purpose register.
- You must never read outside of the `n` bytes of `s` in a way that could cross into unmapped memory and crash. Reading extra bytes from the same aligned chunk that already contains addressable memory of `s` is fine; reading past the end of the buffer's page is not.
- Prototype:

```
void *ft_simd_memchr(const void *s, int c, size_t n);
```



A scalar, byte-by-byte implementation does not satisfy this bonus, even if it returns the correct result. During your defense, be ready to show, in your `.s` source, which instructions perform the parallel comparison.